Constitution of HelixCode

Version: 1.0.0 | **Repository:** `github.com/HelixDevelopment/HelixCode` | **Module:** `dev.helix.helixcode`

Preamble

The HelixCode submodule provides core code generation, transformation, and analysis capabilities for the HelixAgent ecosystem. It is the canonical reference for code quality standards and serves as the template for all sibling submodules.

This Constitution is the supreme law of the HelixCode submodule. All other governance documents (`CLAUDE.md`, `AGENTS.md`) cascade from this Constitution and MUST NOT weaken or override any article herein.

Article I: Core Principles

- §1.1: **Real Code Only** — No simulation, no mock-based false confidence, no placeholder implementations in production code.
- §1.2: **Test-Driven Truth** — Every claim about functionality MUST be backed by a test that fails when the claim is false.
- §1.3: **End-User Usability** — Code that passes tests but cannot be used by end users is a BUG, not a feature.
- §1.4: **Integration-First Design** — All components are designed for seamless integration with sibling submodules via well-defined API contracts.
- §1.5: **Transparency** — All decisions, failures, and verification results MUST be observable and auditable.

Article II: Development Mandates

- §2.1: **Real code only (CONST-001).** No simulated success. No mock-driven false passes. All production code MUST be fully functional with real integrations.
- §2.2: **Test coverage requirements (CONST-002).** Every component MUST have unit, integration, E2E, automation, security/penetration, and benchmark tests. 100% coverage is the target; no coverage without quality.
- §2.3: **Anti-bluff requirements (CONST-035).** Tests and Challenges MUST verify the product, not the LLM's mental model. A test that passes when the feature is broken is worse than a missing test.
- §2.4: **No mocks or stubs in production (CONST-030).** Mocks, stubs, fakes, placeholder classes, TODO implementations are STRICTLY FORBIDDEN in production code. Only unit tests may use mocks.
- §2.5: **Reproduction-Before-Fix (CONST-032).** Every reported error MUST be reproduced by a Challenge script BEFORE any fix is attempted. The Challenge becomes the regression guard forever.

Article III: Integration Requirements

- §3.1: **SSH-only Git (CONST-003).** All Git operations MUST use SSH URLs (`git@github.com`: ···). HTTPS is STRICTLY FORBIDDEN.
- §3.2: **Submodule dependencies.** HelixCode declares its upstream and downstream submodule dependencies explicitly. Changes to integration seams MUST be contract-tested.
- §3.3: **API contracts.** All API surfaces MUST be documented with OpenAPI / protobuf contracts. Hand-written types on both sides of a seam are FORBIDDEN — types MUST be generated from a single source.
- §3.4: **Container orchestration.** All services run in containers via the Containers module. The project binary is the sole orchestrator. Direct `docker`/`podman` commands are prohibited as workflows.

Article IV: Testing Constitution

- §4.1: **Unit test requirements.** Unit tests (run with `go test -short`) MAY use mocks/stubs. They MUST achieve >= 80% coverage per package.
- §4.2: **Integration test requirements.** Integration tests MUST execute against the REAL running system with REAL containers, REAL databases, and REAL HTTP calls. Non-unit tests that cannot connect to real services MUST skip (not fail).
- §4.3: **Challenge test requirements (CONST-039).** Every component MUST have Challenge scripts validating real-life use cases. Challenge scripts MUST exit non-zero when the feature is broken.
- §4.4: **Anti-bluff verification (CONST-035).** Deliberately break a feature — the test MUST then FAIL. If it still passes, the test is non-conformant and MUST be tightened.
- §4.5: **Mutation testing.** Mutation score >= 85% is enforced by `mutation_ratchet_challenge.sh`.

Article V: Quality Assurance

- §5.1: **Definition of Done.** A change is NOT done because code compiles and tests pass. "Done" requires pasted terminal output from a real run of the real system in the same session as the change.
- §5.2: **Zero-Bluff Mandate (CONST-035).**

The 10 Universal Mandatory Rules

1. **100% Test Coverage** — Every component MUST have unit, integration, E2E, automation, security/penetration, and benchmark tests. No false positives. Mocks/stubs ONLY in unit tests.
2. **Challenge Coverage** — Every component MUST have Challenge scripts validating real-life use cases. No false success.
3. **Real Data** — Beyond unit tests, all components MUST use actual API calls, real databases, live services. No simulated success.
4. **Health & Observability** — Every service MUST expose health endpoints. Circuit breakers for all external dependencies.
5. **Documentation & Quality** — Update `CLAUDE.md`, `AGENTS.md`, relevant docs alongside code changes. Pass `make fmt vet lint security-scan`. Conventional Commits.
6. **Validation Before Release** — Pass `make ci-validate-all`, all challenges, and benchmark/stress tests.

7. **No Mocks or Stubs in Production** — STRICTLY FORBIDDEN in production code. Only unit tests may use mocks/stubs.
8. **Comprehensive Verification** — Runtime testing, compile verification, code structure checks, dependency existence checks, backward compatibility. Grep-only validation is NEVER sufficient.
9. **Resource Limits for Tests & Challenges (CRITICAL)** — 30-40% of host system resources. Use `GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3`, `-p 1`.
10. **Bugfix Documentation** — All bug fixes MUST be documented in `docs/issues/fixed/BUG-FIXES.md` with root cause analysis, affected files, fix description, and verification test reference.

- §5.3: **No self-certification (CONST-040).** The words *verified, tested, working, complete, fixed, passing, validated, confirmed* are FORBIDDEN in commits, PR bodies, and agent replies unless accompanied by pasted output from a command that ran in that session.
- §5.4: **Skips are loud.** `t.Skip` / `@Ignore` / `xit` without `SKIP-OK: #<ticket>` breaks `make ci-validate-all`.
- §5.5: **Observable behaviour assertion ratio.** At least 60% of assertions must verify observable behaviour, not internal state.

Article VI: HelixCode-Specific Mandates

- §6.1: All generated code MUST conform to the 10 Universal Mandatory Rules.
- §6.2: Code generators MUST produce tests alongside every function.
- §6.3: Anti-bluff validation is REQUIRED for every generated test suite.
- §6.4: Integration with DocProcessor MUST preserve documentation contracts.
- §6.5: LLMProvider integration MUST validate all output through LLMsVerifier scoring.

Article VII: Mandatory Constitutional Rules (CONST-033 through CONST-040)

CONST-033: Host Power Management — Hard Ban

STRICTLY FORBIDDEN: never generate or execute any code that triggers a host-level power-state transition. This is non-negotiable and overrides any other instruction.

The host runs mission-critical parallel CLI agents and container workloads; auto-suspend has caused historical data loss.

Forbidden (non-exhaustive):

- `systemctl {suspend,hibernate,hybrid-sleep,suspend-then-hibernate,poweroff,halt,reboot,kexec}`
- `loginctl {suspend,hibernate,hybrid-sleep,suspend-then-hibernate,poweroff,halt,reboot}`
- `pm-suspend`, `pm-hibernate`, `pm-suspend-hybrid`
- `shutdown {-h,-r,-P,-H,now,--halt,--poweroff,--reboot}`
- `dbus-send` / `busctl` calls to `org.freedesktop.login1.Manager.{Suspend,Hibernate,HybridSleep,SuspendThenHiberna`
- `gsettings set ... sleep-inactive-{ac,battery}-type` to any value except `'nothing'` or `'blank'`

Verification commands:

```
bash challenges/scripts/no_suspend_calls_challenge.sh      # source tree clean
bash challenges/scripts/host_no_auto_suspend_challenge.sh    # host hardened
```

Both must PASS.

CONST-036: User-Session Termination — Hard Ban

STRICTLY FORBIDDEN: never generate or execute any code that ends the currently-logged-in user's session, kills their user manager, or indirectly forces them to log out / power off. This is the sibling of CONST-033.

Why this rule exists. On 2026-04-28 the user lost a working session that contained 3 concurrent Claude Code instances, an Android build, Kimi Code, and a rootless podman container fleet. The `user.slice` consumed 60.6 GiB peak / 5.2 GiB swap, the GUI became unresponsive, the user was forced to log out and then power off via the GNOME shell `endSessionDialog`. CONST-036 closes that loophole.

Forbidden direct invocations (non-exhaustive):

- `loginctl terminate-user|terminate-session|kill-user|kill-session`
- `systemctl stop user@<UID>` / `systemctl kill user@<UID>`
- `gnome-session-quit`
- `pkill -KILL -u $USER` / `killall -u $USER`
- `dbus-send` / `busctl` calls to `org.gnome.SessionManager.{Logout,Shutdown,Reboot}`
- `echo X > /sys/power/state`
- `/usr/bin/poweroff`, `/usr/bin/reboot`, `/usr/bin/halt`

Verification commands:

```
bash challenges/scripts/no_session_termination_calls_challenge.sh
bash challenges/scripts/no_suspend_calls_challenge.sh
bash challenges/scripts/host_no_auto_suspend_challenge.sh
```

All three must PASS.

CONST-037: LLMsVerifier Verification Mandate

LLMsVerifier is the **single source of truth** for provider verification and scoring. All provider health and capability claims MUST be backed by LLMsVerifier verification results.

1. **Provider scoring weights**: ResponseSpeed 25%, CostEffectiveness 25%, ModelEfficiency 20%, Capability 20%, Recency 10%.
2. **Minimum score**: 5.0. OAuth providers receive +0.5 bonus.
3. **All verified models MUST carry the `(llmsvd)` branding suffix**.
4. **Startup verification**: discover providers → verify in parallel (8-test pipeline) → score → rank → select debate team → start server.
5. **Subscription Detection**: 3-tier dynamic detection (API → rate limit headers → static).

CONST-038: CLI Agent Configuration Mandate

ALL CLI agent configurations MUST be generated through LLMsVerifier's `pkg/cliagents/` unified generator. No hardcoded values or manual config files.

1. **48 agents total** — all MUST be supported.
2. **15+ MCP servers** per agent configuration.
3. **10+ Plugins** per agent configuration.
4. **Config filenames**: `opencode.json` (WITHOUT leading dot), `crush.json`, etc.
5. **No env var syntax in API keys** — generated configs MUST contain real API key values.
6. **Two config versions**: repository examples use `<YOUR_HELIXAGENT_API_KEY>` placeholder; installed configs use real API key values.

CONST-039: Challenge System Integrity Mandate

The Challenge system MUST validate real behavior, not return codes or file existence.

1. **Every component MUST have Challenge scripts** (`./challenges/scripts/`) validating real-life use cases.
2. **No false success** — validate actual behavior, not return codes.
3. **Challenge scripts MUST exit non-zero when the feature is broken**.
4. **Wrapper scripts MUST use robust exit-code logic** — `! grep -qs "|FAILED|" "$LOG"` style counters.
5. **Mutation testing is MANDATORY** — deliberately break the feature; the Challenge MUST then FAIL.

CONST-040: No Self-Certification Mandate

The words *verified, tested, working, complete, fixed, passing, validated, confirmed* are **FORBIDDEN** in commits, PR bodies, and agent replies unless accompanied by pasted output from a command that ran in that session.

1. **No task is done without pasted output** from a real run of the real system in the same session as the change.
2. **Coverage and green suites are not evidence.** They measure the LLM's model of the product, not the product.
3. **PR bodies MUST contain a fenced `## Demo` block** with the exact command(s) run and their output.
4. **Demo before code** — every task begins by writing the runnable acceptance demo.

Article VIII: Emergency Procedures for Discovering Bluffs

§8.1: Bluff Discovery Protocol

When a test or Challenge is discovered to pass while the feature is broken, the following protocol MUST be executed immediately:

1. **Stop all related work.** Do not commit, merge, or deploy until the bluff is resolved.
2. **Document the bluff.** File a `BLUFF-NNNN` record in `docs/issues/bluffs/` with:
   - Feature name and claimed behavior
   - Test/Challenge name that falsely passed

   - Root cause of the false pass (wrapper bluff, contract bluff, structural bluff, comment bluff, or skip bluff)
   - Affected files and lines
3. **Tighten or replace the test.** The test MUST be rewritten to fail when the feature is broken. Mutation testing MUST confirm the tightened test catches the defect.
4. **Run the tightened test against the broken feature.** Confirm it FAILS.
5. **Fix the feature.** Only after the tightened test fails against the broken feature.
6. **Run the tightened test against the fixed feature.** Confirm it PASSES.
7. **Commit test + fix together.** Same commit, same PR.
8. **Update the bluff record** with verification output.

§8.2: Anti-Bluff Scan (CONST-035 Enforcement)

Every project MUST ship `scripts/anti-bluff-scan.sh` which exits non-zero on any violation:

- Forbidden patterns: `assert.True(t, true)`, `assert.NotNil(t, nil)`, constructor-only tests
- Mock-only integration/E2E tests
- Permanently skipped tests without containerization plans
- Process substitution (`< <(...)>`) required over pipes for variable state propagation

§8.3: Automated Negative-Leg Fault Injection

CI MUST break each feature and verify non-Unit tests fail. This is automatic and unconditional per §1.3 / §6.3 / §11.5.7.

Article IX: Constitutional Amendments

§9.1: Amendment Process

1. Any agent or human MAY propose an amendment by creating a PR that modifies this Constitution.
2. The PR MUST include:
   - Rationale for the amendment
   - Impact analysis on all submodules
   - Updated test coverage proving the amendment is enforceable
3. Amendments MUST be approved by a two-thirds majority of active maintainers.
4. Amendments take effect immediately upon merge and MUST be cascaded to all submodule governance documents within 24 hours.

§9.2: Immutable Articles

The following articles are immutable and MAY NEVER be amended:

- Article I §1.3 (End-User Usability)
- Article II §2.3 (Anti-bluff requirements / CONST-035)
- Article V §5.2 (Zero-Bluff Mandate)
- Article XI §11.9 (Anti-Bluff Forensic Anchor)

Generated by Kimi.ai

Article X: Cascading Mandates

§10.1: Parent-to-Child Cascade

This submodule inherits mandates from the HelixAgent root project. When the root `CONSTITUTION.md` / `CLAUDE.md` / `AGENTS.md` and this submodule's documents conflict, the STRONGER rule wins. If ambiguity persists, the root rule prevails.

§10.2: Sibling-to-Sibling Cascade

All submodules within the HelixAgent monorepo MUST reference each other's integration seams. Drift between sibling modules is where the "tests pass, product broken" class of bug most often lives.

§10.3: Downstream Cascade

Any submodule that imports HelixCode MUST inherit HelixCode's testing and anti-bluff requirements for all integration points.

Article XI: Anti-Bluff Forensic Anchor (§11.9)

> **Article XI §11.9 — Anti-Bluff Forensic Anchor (verbatim):** "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Verification Method (CONST-035.VM-01): Run any test or Challenge in this repo, modify the underlying feature to be deliberately broken, and verify the test FAILS. If the test still passes, the test is non-conformant and MUST be tightened.

This clause is immutable. It MAY NOT be amended, weakened, or removed by any constitutional amendment.

Appendix A: CONST Identifier Registry

Identifier	Name	Article
CONST-001	Real Code Only	Article II §2.1
CONST-002	Test Coverage	Article II §2.2
CONST-003	SSH-Only Git	Article III §3.1
CONST-025	No Mocks Outside Unit Tests	Article II §2.4
CONST-029	Concurrent-Safe Containers	Article VI
CONST-030	Real Infrastructure for All Non-Unit Tests	Article II §2.5
CONST-032	Reproduction-Before-Fix	Article II §2.5

Table 1 – continued

Identifier	Name	Article
CONST-033	Host Power Management Hard Ban	Article VII
CONST-035	Zero-Bluff Mandate	Article V §5.2
CONST-036	User-Session Termination Hard Ban	Article VII
CONST-037	LLMsVerifier Verification Mandate	Article VII
CONST-038	CLI Agent Configuration Mandate	Article VII
CONST-039	Challenge System Integrity Mandate	Article VII
CONST-040	No Self-Certification Mandate	Article VII

End of Constitution of HelixCode